Research Question

Can editing audio distort someone's voice enough to make it effectively anonymous, and how do we model the trade-off between privacy and intelligibility in anonymized speech data?

Brief overview

Nowadays, most of us have voice assistants. Whether it's Siri and/or Alexa (fitting…), our voices are used daily to administer everyday commands. How much protection is put into making our voices anonymous before being processed by our smart devices? In this experiment, we aim to see how many levels of abstraction it takes before our voices are unrecognizable. Will the trade-offs be worth it? How much can we manipulate audio before it is incomprehensible? And is anonymity truly achievable when related to audio?

W0: Initial explanation, inputs & outputs

Our problem is to see how anonymous voice samples can be before we lose the meaning of the audio, and what the trade-off is when it comes to audio being private and usable. Our inputs ended up being voice recordings from an OpenSLR database which we will talk about more in W2**.** We ran our database through code that manipulated it in pitch, added noise and formant, meaning the resonance of the vocal tract. We also took every combination of those three things, ultimately manipulating each audio file 7 times. We had 20 different speakers and each of those speakers had 15+ audio snippets. We ran the code 3 times, each time took over 30 mins and produced about 2.5 gigs of manipulated audio data. (our machines were not happy with us). With the original and manipulated audio clips, we were able to produce scores "Accuracy" and "Intelligibility" (Privacy vs Accuracy)as our outputs. Our "Accuracy" score is represented by a machine learning model's accuracy in predicting which out of the 20 users a given clip belongs to. Our "Intelligibility" score is represented by an Automatic Speech Recognition (ASR) tool's Word Error Rate (WER).

W1: Existing research on privacy procedures for popular voice assistants, Motivation

After doing some preliminary research, specifically on Amazon Alexa protocols, we couldn't find the specific ways that our audio is being recorded. Most Amazon websites say that yes, your voice is being recorded and uploaded to AWS, and that it is also used as a way to improve accuracy. This text usually also comes with a way of deleting stored voice recordings… So is privacy not something that people are worried about anymore? Our voice recordings in raw, unedited form is something that can be very powerful and usually can not be drastically changed. Voice is a very identifiable factor of ourselves and we should know if it can be protected and how it is being stored.

W2: Data sources

Collecting data was a huge bottleneck in our last project and we didn't want to repeat that. As outlined in our proposal, our original idea was to record our own voices. But even with the larger time frame, at the end of the day, collecting the data isn't the point of this project. After wasting a little too much time trying to manually build our dataset, we realized that it would give us more

time to focus on the analysis and parts that matter if we outsourced to a publicly available dataset. This was when we started looking for databases online.

Finding a dataset

Who knew finding a large, public, free, high-quality dataset of people's voices would be so hard! After research we decided that we needed something that was:

1. Large enough to produce informative predictions when trained upon
2. Free use, modify, and distribute
3. Contained enough English speakers/accents for our ASR to perform as optimally as possible.

We first looked at CelebVox, which is a compilation of MP4s extracted from YouTube videos of celebrities [1]. It had enough speakers, but even though CelebVox distributes its metadata under “research use” terms, YouTube videos themselves are not freely redistributable. You can’t legally package the extracted audio, upload it anywhere, or share it as part of the project. On top of that everything was in MP4, so we’d have to convert the whole thing to WAV anyway before doing any transformations. So, we pivoted and settled on the LibriSpeech ASR corpus from Open Speech and Language Resources (OpenSLR) [2]. The audio consists of people reading passages from books, and while that isn't the same as a command to a voice assistant, it still reflected natural spoken English enough that we thought it would work for our testing. Also, LibriSpeech is released under the Creative Commons Attribution 4.0 International (CC by 4.0) license, which means we are legally allowed to download, modify, use for research, and share it with citation.

C3: Audio Transformation (voice_changer.py)

Surprisingly, the code to manipulate the audio was one of the simplest things we did for this project. Overall the voice_changer.py file is 88 lines long of which some are to make sure that the audio files output to the correct directory. For it to be easier for the model to understand we had to change the code multiple times. Ultimately we ended up with a folder for each combination of manipulation and within that we had a folder for each speaker so each audio file would be organized accordingly.

```
# funcs
def do_pitch(x):
    return librosa.effects.pitch_shift(x, sr=sr, n_steps=-3)

def do_form(x):
    return formant_shift(x, sr)

def do_noise(x):
    s = librosa.effects.time_stretch(x, rate=1.0005)
    n = np.random.normal(0, 0.05, len(s))
    return s + n
```

For two of the functions, pitch and noise it was very simple to implement. However, testing was the most complicated. It took a lot of playing around with them to make sure they weren't immediately intelligible. Overall pretty simple, just annoying to find a happy medium of clear change and still being able to understand the speaker.

```
#------------------------
def formant_shift(audio, sr, ratio=1.3):
    f0, sp, ap = pw.wav2world(audio.astype(np.float64), sr)
    new_bins = int(sp.shape[1] * ratio)

    sp_shifted = resample(sp, new_bins, axis=1)
    sp_shifted = sp_shifted[:, :sp.shape[1]] if new_bins > sp.shape[1] else np.pad(
        sp_shifted, ((0,0),(0, sp.shape[1]-new_bins)), mode="edge"
    )

    sp_shifted = np.ascontiguousarray(sp_shifted)
    shifted = pw.synthesize(f0, sp_shifted, ap, sr)
    return shifted / np.max(np.abs(shifted))
```

Then we have the formant shifter using pyworld. For the ratio, anything greater than 1 makes the voices sound younger and since pitch was being used to alter the voice to be deeper we decided to have formants do the opposite. Then we divided the audio clip and transformed it into a 64b float because pyworld requires that. Then we manipulate the individual components, put them back together and make sure the audio clip is normalized, minimizing

C4: Speaker Recognition Model (train_speaker_cnn.py & eval_speaker_cnn.py)

The speaker recognition model is what we used to measure how well an attacker could still identify the speaker after we applied voice transformations if they obtained access to the user's unaltered voice recordings. In our proposal feedback, we were pointed to a survey by Lukic et al that reviewed different ML approaches for voice recognition [3]. A lot of previous studies used Convolutional Neural Networks (CNNs) on spectrogram inputs, and discussed how CNNs can pick up on acoustic patterns people leave in their speech. We decided to follow in their footsteps and trained on the unaltered recordings.

<u>train_speaker_cnn.py</u>

This script goes through every clean audio clip, turns it into a mel-spectrogram, and trains the model to match those features to the correct speaker ID. After each epoch it checks how well the model does on a validation split, and whenever it reaches a new best accuracy it saves a checkpoint. That checkpoint is what we use later to test how the transformations affect the model's ability to identify the speaker.

```
1  import random
2  from pathlib import Path
3  import librosa
4  import numpy as np
5  import torch
6  import torch.nn as nn
7  import torch.nn.functional as F
8  import torch.optim as optim
9  from torch.utils.data import Dataset, DataLoader
```

Libraries used

We used random for shuffling files for our train/test split, numpy for normalization and padding, librosa for loading .wav files and mel-spectrograms, and Pytorch for the CNN training (tensor operations, layers, loss, optimizer, etc)

```python
#small CNN for speaker classification
class SimpleCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        #define layers (conv, pool, fc, dropout)
        #3 conv layers + pooling
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.dropout = nn.Dropout(0.3)
        #fully connected layers
        self.fc1 = nn.Linear(64 * 5 * 10, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        b, c, h, w = x.shape
        x = x.view(b, -1)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x
```

SimpleCNN(nn.Module) & forward(self, x)

The CNN model takes in a mel-spectrogram for each utterance and passes it through 3 convolution-pooling blocks to extract meaningful features from the audio. Then it flattens everything and runs it through 2 fully connected layers to classify which speaker it belongs to.

Expectedly, it was very good at classifying who was speaking on the clean audio and reached about 98% accuracy in training and 96% in evaluation on that same dataset. We then ran it on the transformed versions of the same data to compare how the accuracy dropped, which measures how strong the transformation was at anonymization.

Eval_speaker_cnn.py

```
    #walk thru root & all subfolders
    for dirpath, _, filenames in os.walk(root):
        dirpath = Path(dirpath)

        for fn in filenames:
            if Path(fn).suffix.lower() not in audio_exts:
                continue

            file_count += 1
            filepath = dirpath / fn
            true_spk = filepath.parent.name  #ex. spk001
            #extract features & send thru model
            feat = extract_features(filepath, sr, n_mels, max_frames).unsqueeze(0).to(device)

            with torch.no_grad():
                logits = model(feat)
                probs = torch.softmax(logits, dim=1)
                pred_idx = int(torch.argmax(probs, dim=1))
                pred_spk = label_to_speaker[pred_idx]
                conf = float(probs[0, pred_idx])

            correct = int(pred_spk == true_spk)
            results.append([str(filepath), true_spk, pred_spk, conf, correct])
```

This script loads the saved CNN checkpoint and all the preprocessing settings it used during training, and turns every .wav file into the same mel-spectrogram format the model expects. Then it loops through each audio file, grabs the correct speaker from the folder name, feeds the features into the CNN, and logs what the model thinks the speaker is along with the confidence. All of that gets saved into a CSV so we can compare the true vs. predicted label later, and it calculates the overall accuracy.

```
(venv) PS C:\Users\hp\Downloads\1657\p2-gabexa-proj2> python code/eval_speaker
.py --root code/data --out results/full_eval1.csv
>>
found 1043 files
acc: 96.26%
results saved at: C:\Users\hp\Downloads\1657\p2-gabexa-proj2\results\full_eval
```

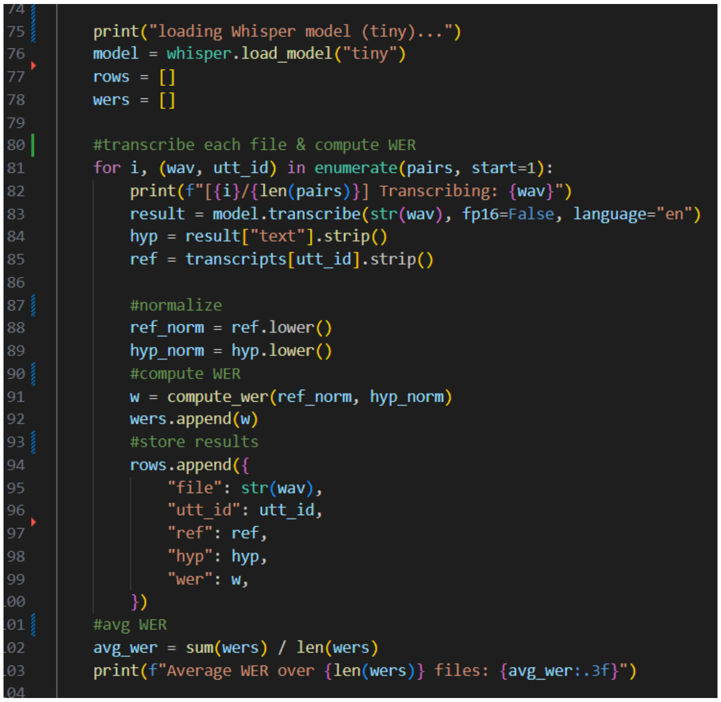
Accuracy for model trained on unmodified speaker voices

```
PS C:\Users\hp\Downloads\1657\p2-gabexa-proj2> python code/eval_speaker_cnn.py
found 1043 files
acc: 23.01%
results saved at: C:\Users\hp\Downloads\1657\p2-gabexa-proj2\results\form.csv
```

Accuracy for model trained on speaker voices with only form feature changed

C5: Whisper recognition for intelligibility (asr_intelligibility_whisper.py)

Like we mentioned in our proposal, we planned to use a speech recognition API like Whisper or Google Speech to measure how intelligible our transformed audio remained. We wanted to try Google, but it costs money and even if it would have only been a few dollars I've already spent enough on my degree… OpenAI's Whisper didn't require any API keys or billing cards so it was the second best choice.



```
    print("loading Whisper model (tiny)...")
    model = whisper.load_model("tiny")
    rows = []
    wers = []

    #transcribe each file & compute WER
    for i, (wav, utt_id) in enumerate(pairs, start=1):
        print(f"[{i}/{len(pairs)}] Transcribing: {wav}")
        result = model.transcribe(str(wav), fp16=False, language="en")
        hyp = result["text"].strip()
        ref = transcripts[utt_id].strip()

        #normalize
        ref_norm = ref.lower()
        hyp_norm = hyp.lower()
        #compute WER
        w = compute_wer(ref_norm, hyp_norm)
        wers.append(w)
        #store results
        rows.append({
            "file": str(wav),
            "utt_id": utt_id,
            "ref": ref,
            "hyp": hyp,
            "wer": w,
        })
    #avg WER
    avg_wer = sum(wers) / len(wers)
    print(f"Average WER over {len(wers)} files: {avg_wer:.3f}")
```

It loops over each audio file, runs Whisper-tiny to get a predicted transcript, grabs the matching reference transcript from our CSV, and normalizes both to avoid tiny casing/punctuation mismatches. Then it computes the WER between the two and logs the results. After processing everything, it calculates the average WER to see how much the transformation hurt intelligibility overall.

Like the audio transformations this was very hard on our machines and time. For our ASR intelligibility with Whisper, originally we were transcribing every .wav file against the transcript which took upwards of 40 minutes. So we had to pivot by taking a subset of 400 .wav files equally distributed among the 20 users which only took around 6 minutes for each transformation/group of transformations since each took around 2-3 seconds to transcribe.

W6: Results

For the score we are looking for two things, low accuracy and low intelligibility. So did the scoring with good ol ((1-accuracy)+(1-ASR))/ 2 and the highest score wins. We know this isn't exactly perfect because this would mean that accuracy and quality are placed at the same importance, also making it possible for a model like Pitch_noise that is completely intelligible, still isn't the worst on our ranking system. So we know it isn't perfect but it's the best we could think of

Type of audio	ML Speaker Recognition (Accuracy)	Average ASR Word Error Rate (Intelligibility)	Score	Ranking
Original	**96.0%**	**0.19**	**0.425**	**5**
Form	**23.0%**	**0.26**	**0.755**	**1**
Pitch	**40.64%**	**0.60**	**0.4968**	**4**
Noise	**34.10%**	**0.62**	**0.5195**	**3**
Pitch_form	**67.30%**	**0.79**	**0.2685**	**8**
Form_noise	**12.98%**	**0.65**	**0.6101**	**2**
Pitch_noise	**29.18%**	**1.017**	**0.3456**	**7**
Pitch_noise_form	**23.34%**	**.97**	**0.3983**	**6**

These results feel a little bittersweet. After listening to the audio recordings, yes they do seem like a completely different person however. The voice changing code that we used is under a hundred lines long, how hard would it be to reverse this process. However, we are very happy with our results. We think that it was a very balanced data set with different accents and genders so we are very proud of the output we got. :)

Results that made us question what was going on:

- Pitch_forms accuracy is very high compared to the others. This is because formant changes the tone of voice to be younger while pitch changes the audio to be deeper so the two are trying to cancel each other out.
- Why is pitch_noise intelligibility above 1? We honestly thought we broke something when we saw this number. However it made perfect sense once we investigated a little deeper. Although the whisper module outputs in percentage format, there is no upper limit to this number. This is because the denominator is the number of words. So if the audio only had 2 words but the system made 10 mistakes the output would be 5.0 (or 500%). This just means our pitch noise audio snippets are absolute gibberish.
- For pitch_noise_form we just thought it was interesting to mention that the only reason that the intelligibility is slightly lower is because once again, pitch and formant cancel each other out enough where the model can somewhat understand the clips.

Example of a mistake in formant transformation:

Original Transcript	Formant ASR transcript with Whisper
HE MEANT TO HAVE IT ONE OF THE BEST IN THE WORLD SO HE CALLED AN OFFICER INTO HIS COUNCIL CHAMBER AND SAID NOW TAKE PLENTY OF TIME TO LOOK ABOUT IN THE DIFFERENT COUNTRIES HAVE ALL THE MEN YOU WANT TO HELP YOU	**In it to** have it one of the best in the world, so he caught an officer into his **couch or** chamber and said. Now take plenty of time to look about in the different countries. Have all the **many we** want to help you

Overall, whisper was very good at detecting keywords and somewhat preserved the meaning. Again if someone were to intercept the whisper transcript with ill intent the true meaning could definitely be extracted like we talked about in class.

W7: Shortcomings, future improvements, pivots, and reflection

We had so many dependency issues. Alexa is working on Windows and Gaby on Mac so all of the terminal commands we had to do were different as well (that took us a while to figure out). A huge problem that we had was the size of our data set. It took so long to process everything through the cnn, voice changer and whisper model simply because we had thousands of samples. It was also a huge problem to get the files to each other, our computers were moving so slow and had to be restarted multiple times because such a simple task like compressing files or moving them from folder to folder took forever.

We didn't take many pivots per say unlike last time, besides our initial change from collecting our own data to using a publicly available dataset and some difficulties with Google Speech which made us change to Whisper. I think we chose something which was more straightforward and had less room to play around with other options. However the steps we did choose were a lot more time consuming. As mentioned before, our dataset was huge because we didn't want to have the same problem as last time and not having sufficient data to train the model. As mentioned in the sections above, we struggled a lot more on different things like playing around with the audio manipulation, formatting the data for the model, and getting the Whisper model to work. On top of that everytime we decided to run/test something, it would take 30+ mins per speaker.

Some future improvements we could make would be to add more audio manipulation features, for this project we thought it was best to stick to 3 and all of the combinations they had to keep it manageable. As said before these 7 manipulations produced 2.5 gb of data which at times was overwhelming. However with more powerful machines, time, and storage we would've loved to have played around with different audio manipulation techniques.

Overall we loved doing this project! It was definitely more of a pain than the other one since steps just took longer overall. We had both never had experience with audio manipulation, but just like we were unfamiliar at the start of our last project I think choosing a topic based on our natural interest and not just what we thought would work best really helped us stay motivated and feel that we gained skills we can use moving forward. We were also able to build off of the

knowledge we had gained in ML with our last project, so I'm glad it was suggested that we continue with that approach in our proposal feedback.

Bibliography

[1] VoxCeleb. https://www.robots.ox.ac.uk/~vgg/data/voxceleb/

[2] Openslr.org. https://www.openslr.org/12/

[3] N. H. Tandel, H. B. Prajapati and V. K. Dabhi, "Voice Recognition and Voice Comparison using Machine Learning Techniques: A Survey," 2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2020, pp. 459-465, doi: 10.1109/ICACCS48705.2020.9074184. keywords: {Speaker recognition;Feature extraction;Speech recognition;Artificial neural networks;Machine learning;Spectrogram;Mel frequency cepstral coefficient;voice comparison;speaker recognition;deep learning;Siamese NN}, https://ieeexplore.ieee.org/abstract/document/9074184